



Servidor Echo em TypeScript

Atividade 1.2 - Sistemas Operacionais (Prof. L. A. Minora)

Equipe: **Antônio Irineu**, **Maycon Alexsander** e **Lucas Vinicius**

Sumário

1. Linguagem e Grupo
2. Sumário
3. Bibliotecas Usadas
4. Comparativo: Com e Sem Thread
5. Dificuldades
6. Observações do Grupo

Bibliotecas Usadas

- Foi utilizado o **Deno**, um runtime moderno para TypeScript.
- A comunicação TCP foi feita utilizando a API nativa do próprio runtime:
 - `Deno.listen()`: Para abrir a porta do servidor.
 - `Deno.connect()`: Para a conexão do cliente.
 - `Deno.Conn`: Interface nativa para operações de leitura e escrita.



Comparativo: Sequencial vs. Concorrente

Sem Thread / Sequencial

- O uso de `await` forçou o bloqueio do fluxo.
- O S.O. aceitou as conexões físicas TCP, mas a aplicação reteve os clientes em uma fila.
- Os clientes foram processados um a um.

```
mayco@PC-Maycon MINGW64 ~/OneDrive - IFRN/3º PERÍODO/Sistemas Operacionais/2026-1-atividade-1-2 (main)
$ docker compose -f docker-compose-sequencial.yml logs -f servidor
```

Com "Thread" / Concorrente

- Remoção do `await` para delegar a rotina ao Event Loop.
- A thread principal permaneceu liberada.
- Atendimento simultâneo dos 10 clientes sem necessidade de múltiplas threads do S.O.

```
mayco@PC-Maycon MINGW64 ~/OneDrive - IFRN/3º PERÍODO/Sistemas Operacionais/2026-1-atividade-1-2 (main)
$ docker compose -f docker-compose-sequencial.yml logs -f servidor
```

Dificuldades



Compreender a natureza Single-Thread do TypeScript e como ele lida com concorrência sem criar instâncias de Threads reais do Sistema Operacional.



Configurar o Docker Compose para orquestrar o servidor e 10 réplicas simultâneas de clientes em uma mesma rede virtual.

Demais Observações

Em operações como conexões de rede, a abordagem **assíncrona** (async/await) mostrou-se extremamente eficiente.

Ao **evitar a criação de threads** de S.O. (modelo 1:1), eliminamos o custo computacional com a troca de contexto (Context Switch) pelo escalonador.

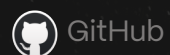
A passagem de **sequencial para concorrente** exigiu a alteração de apenas uma linha de código:

```
// SEQUENCIAL
for await (const conexao of listener) {
  // O 'await' trava o loop até o cliente sair
  await tratarConexao(conexao);
}

// CONCORRENTE
for await (const conexao of listener) {
  // Sem o 'await', o loop delega a rotina e continua livre
  tratarConexao(conexao);
}
```

Obrigado!

Dúvidas?



GitHub – mayconalex/2026-1-at...

Atividade avaliativa do 1o bimestre de
2026.1 da disciplina de Sistemas...